

팍리스 실무형 일경험 프로그램

SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



8일차

01 Simulator Zone – Control App 구현

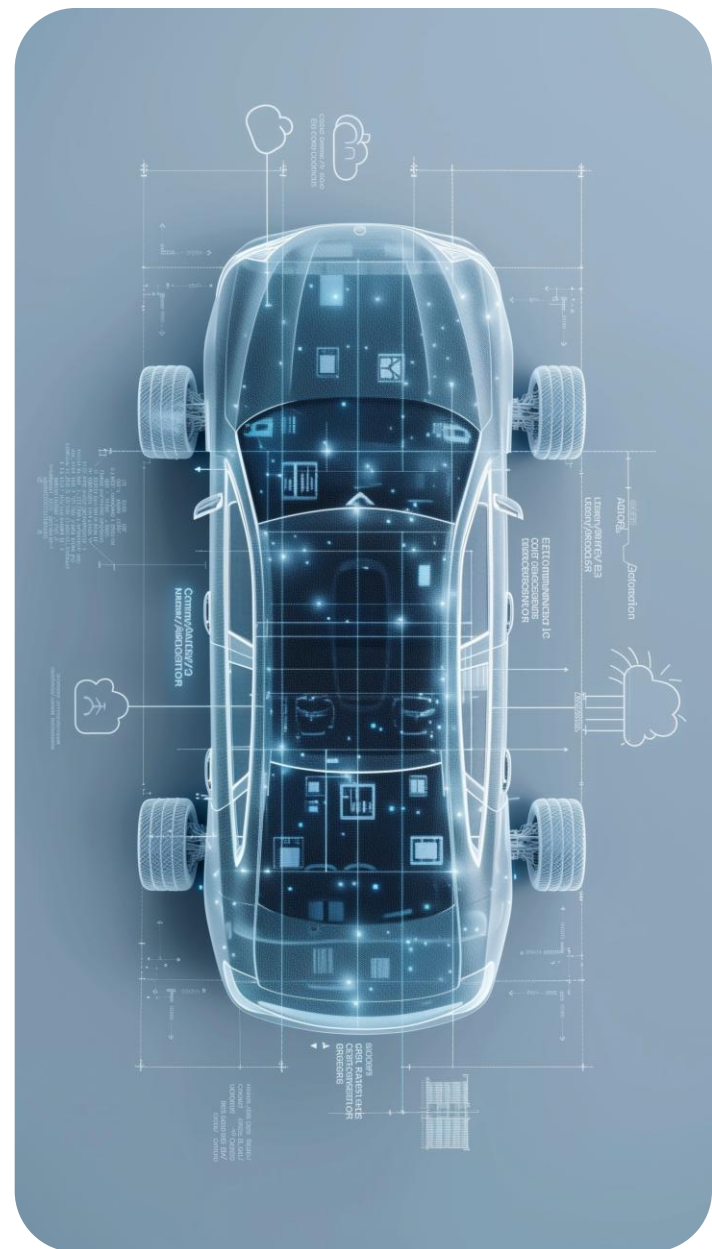
Telechips  **TOPST**



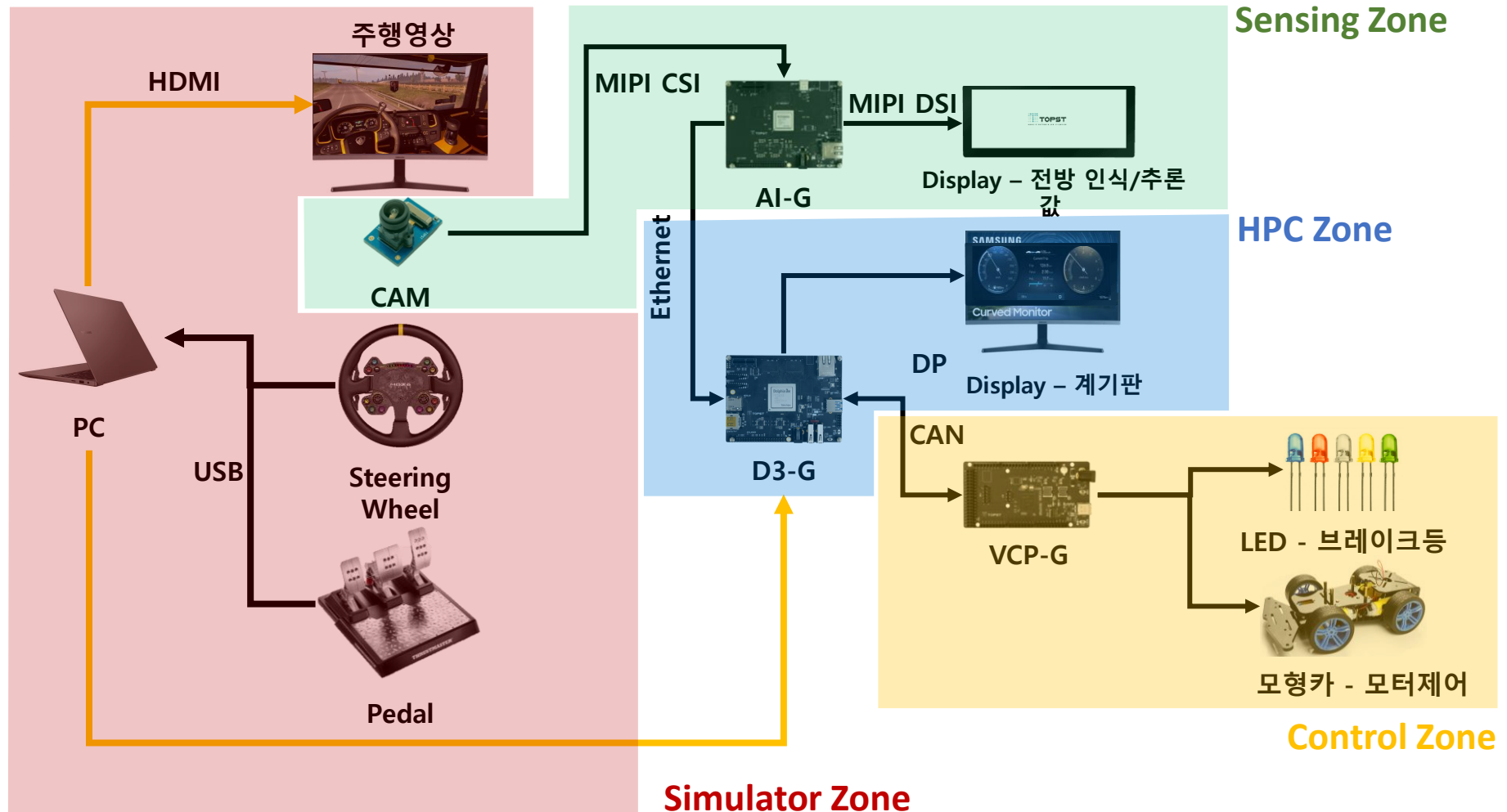
목차

Table of Contents

D3-G A72 Control App 이해	01
Control App 코드 작성	02
Control App 실행	03
Control App 추가 기능 구현	04



Zonal 기반 차량 인터랙션 시스템-전체 시나리오

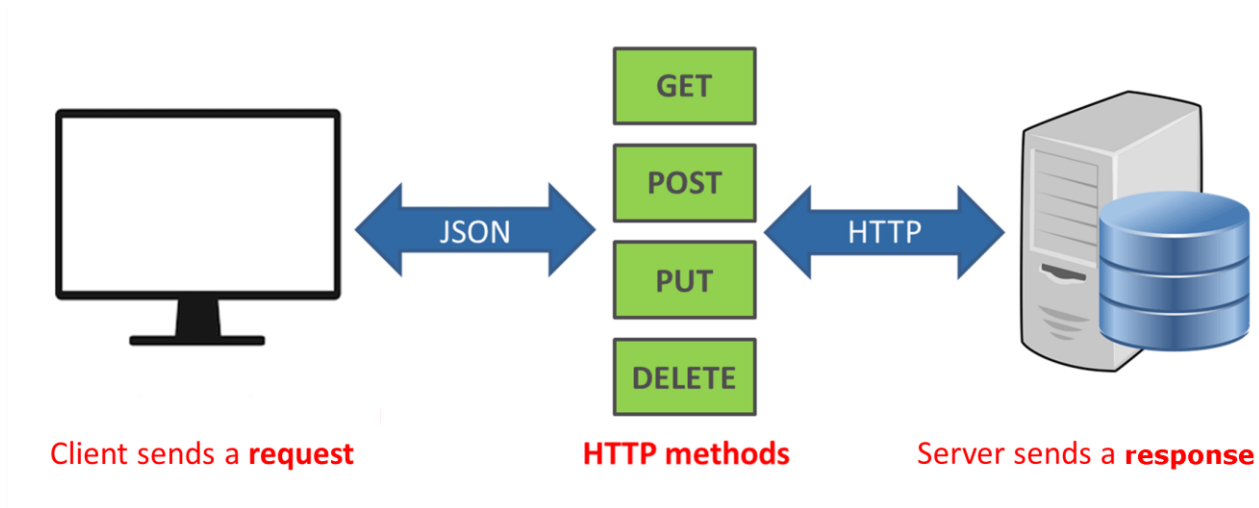


D3-G A72 Controller App 이해

- REST(Representational State Transfer)
 - 자원을 이름으로 구분하여 해당 자원의 상태를 주고 받음
- 특징
 1. HTTP URI를 통해 자원을 명시
 2. HTTP Method(POST, GET, PUT, DELETE, PATCH)를 해당 자원에 적용
- 구성 요소
 - 자원 : HTTP URL
 - 자원에 대한 행위 : HTTP Method
 - 자원에 대한 행위의 내용 : HTTP Message Pay Load

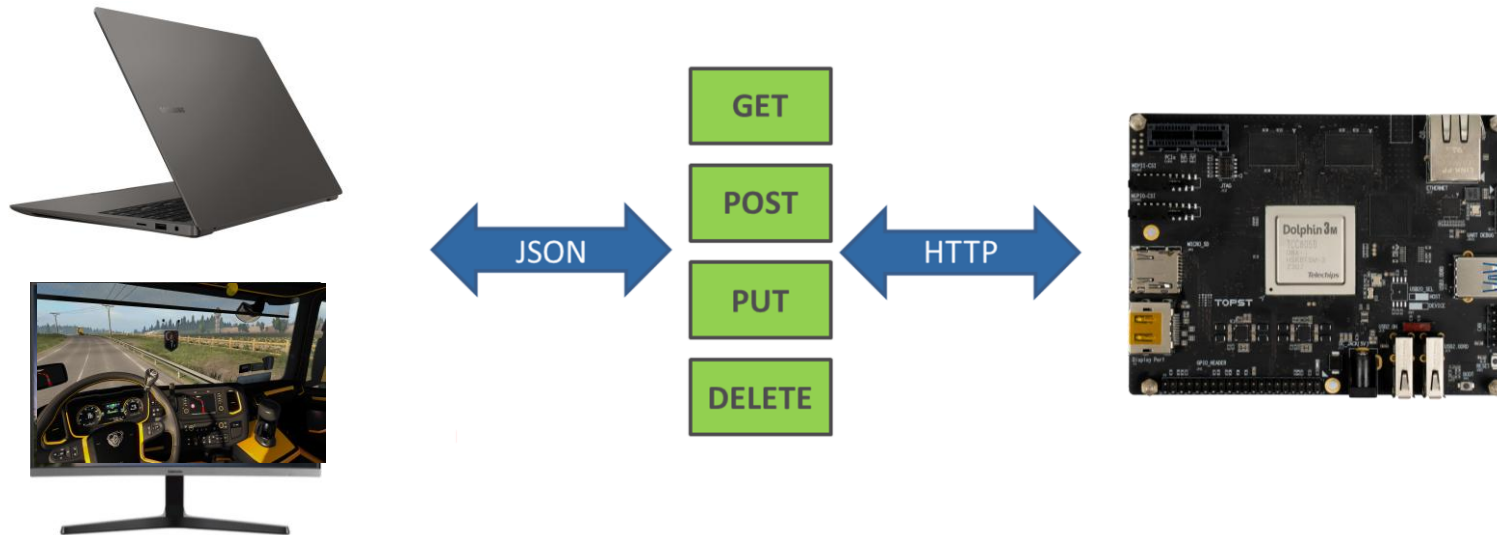
D3-G A72 Controller App 이해

- REST API
 - 웹에서 HTTP URI를 통해 자원을 HTTP 규칙으로 주고 받는 방식
 - JSON으로 데이터를 교환



동작 구조

- Simulator Zone
 - Telemery의 HTTP URI를 통해 차량의 정보 수집
 - JSON 형식으로 데이터를 수집



Control App 코드 작성

- 파일 수정 권한 부여
- `$ cd Fabless_Education_blink/Step1`
- `$ sudo chmod 777 *`

Control App 코드 작성

- D3-G 보드에서 편집기 실행

\$ geany

- File -> Open -> Fabless_Education -> Step1 -> Data_parsing.py 선택
- 빈칸 코드 작성 진행

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

# =====
# Library Imports (환경에 맞게 유지)
#
# =====
import json, time, http.client
import argparse
import threading
from urllib.parse import urlparse
from dataclasses import dataclass, field
from typing import Dict, Any, Optional
```

```
# =====
# Configuration & Constants
# =====

@dataclass
class Config:
    """Application Configuration"""
    TELEMETRY_URL: str =
        "http://10.10.129.118:25555/api/ets2/telemetry"

    # Frequencies (Hz)
    POLL_HZ: int = 60
    PRINT_HZ: int = 30
```


Control App 코드 작성

- D3-G 보드에서 편집기 실행
\$ geany
- Control App 코드 작성

```
# =====  
# Main Monitor Class  
# =====  
  
class ETSTelemetryMonitor:  
    def __init__(self, config: Config):  
        self.cfg = config  
        self.running = threading.Event()  
        self.lock = threading.Lock()  
        self.threads = []  
  
        # State Data (Shared Resources)  
        self.telemetry_data = {  
            "rpm": 0.0, "speed_kmh": 0.0, "fuel_l": 0.0, "steer": 0.0,  
            "park": False, "left_blinker": False, "right_blinker": False,  
            "emergency": False, "headlights": False  
        }
```

Control App 코드 작성

- D3-G 보드에서 편집기 실행
\$ geany
- Control App 코드 작성

```
def __enter__(self):  
    """Start"""  
    print(f"[SYS] Connecting to Telemetry Server: {self.cfg.  
TELEMETRY_URL}")  
    self.running.set()  
    return self  
  
def __exit__(self, exc_type, exc_val, exc_tb):  
    """Resource Cleanup"""  
    print("[SYS] Shutting down...")  
    self.running.clear()  
  
    for t in self.threads:  
        t.join(timeout=1.0)  
    print("[SYS] Shutdown complete.")
```

Control App 코드 작성

- D3-G 보드에서 편집기 실행

\$ geany

- Control App 코드 작성

```
def _parse_telemetry(self, js: Dict) -> Dict[str, Any]:
    """Safely parse JSON to internal state format"""
    def get_val(path, type_cast, default):
        cur = js
        try:
            for key in path.split("."):
                cur = cur[key]
            return type_cast(cur)
        except (KeyError, TypeError, ValueError):
            return default
    truck = "truck"
    left_on = get_val(f"{truck}.blinkerLeftOn", bool, False)
    right_on = get_val(f"{truck}.blinkerRightOn", bool, False)
    return {
        "rpm": get_val(f"{truck}.engineRpm", float, 0),
        "speed_kmh": get_val(f"{truck}.speed", float, 0),
        "fuel_l": get_val(f"{truck}.fuel", float, 0.0),
        "steer": get_val(f"{truck}.gameSteer", float, 0.0),
        "park": get_val(f"{truck}.parkBrakeOn", bool, False),
        "left_blinker": get_val(f"{truck}.blinkerLeftActive", bool, False),
        "right_blinker": get_val(f"{truck}.blinkerRightActive", bool, False),
        "emergency": (left_on and right_on),
        "headlights": get_val(f"{truck}.lightsBeamLowOn", bool, False),
    }
```

Control App 코드 작성

- D3-G 보드에서 편집기 실행

\$ geany

- Control App 코드 작성

```
def _worker_http_poller(self):
    """Thread: Fetches data from ETS2 Telemetry Server"""
    u = urlparse(self.cfg.TELEMETRY_URL)
    path = u.path or "/"
    if u.query: path += "?" + u.query

    headers = {"User-Agent": "ETS2-Client/2.0", "Connection": "keep-alive"}
    period = 1.0 / self.cfg.POLL_HZ

    print(f"[NET] Poller started. Target: {u.hostname}:{u.port}")

    while self.running.is_set():
        start_time = time.monotonic()
        conn = None
        try:
            conn = http.client.HTTPConnection(u.hostname, u.port or 80, timeout=1)
            conn.request("GET", path, headers=headers)
            resp = conn.getresponse()
```

Control App 코드 작성

- D3-G 보드에서 편집기 실행

\$ geany

- Control App 코드 작성

```

        if resp.status == 200:
            data = resp.read()
            js = json.loads(data.decode("utf-8", "ignore"))
            new_state = self._parse_telemetry(js)

            with self.lock:
                self.telemetry_data.update(new_state)
        else:
            # Simple backoff on error
            time.sleep(0.5)

    except Exception as e:
        # print(f"[NET] Polling Error: {e}") # Verbose error suppression
        time.sleep(1.0)
    finally:
        if conn: conn.close()

    # Accurate Timing Loop
    elapsed = time.monotonic() - start_time
    sleep_time = period - elapsed
    if sleep_time > 0:
        time.sleep(sleep_time)

```

Control App 코드 작성

- D3-G 보드에서 편집기 실행

\$ geany

- Control App 코드 작성

```
def _worker_printer(self):
    """현재 상태를 화면에 출력하는 스레드"""
    print("[SYS] Printer started.")
    period = 1.0 / self.cfg.PRINT_HZ

    while self.running.is_set():
        start_time = time.monotonic()

        with self.lock:
            d = self.telemetry_data
            # 한 줄로 깔끔하게 출력 (f-string 포매팅 활용)
            # \r을 사용하여 같은 줄에서 갱신되도록 함
            status_str = (
                f"\r[ETS2] "
                f"RPM: {d['rpm']:6.1f} | "
                f"Speed: {d['speed_kmh']:5.1f} km/h | "
                f"Fuel: {d['fuel_1']:5.1f} L | "
                f"Steer: {d['steer']:5.2f} | "
                f"Light: {'HEAD' if d['headlights'] else 'OFF '} | "
                f"Signal: {'L' if d['left_blinker'] else '_'}{'R' if d['right_blinker'] else '_'} "
            )
        print(status_str, end="", flush=True)
```

Control App 코드 작성

- D3-G 보드에서 편집기 실행
\$ geany
- Control App 코드 작성

```
# 주기 맞추기
elapsed = time.monotonic() - start_time
sleep_time = period - elapsed
if sleep_time > 0:
    time.sleep(sleep_time)
```

Control App 코드 작성

- D3-G 보드에서 편집기 실행

\$ geany

- Control App 코드 작성

```
def start(self):
    """Start all worker threads"""
    workers = [
        self._worker_http_poller,
        self._worker_printer
    ]

    for func in workers:
        t = threading.Thread(target=func, daemon=True, name=func.__name__)
        t.start()
        self.threads.append(t)

    print("[SYS] All threads started.")

    # Main thread loop
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("\n[SYS] KeyboardInterrupt detected.")
```


Control App 코드 작성

- D3-G 보드에서 편집기 실행

\$ geany

- Control App 코드 작성

```
# =====  
# Entry Point  
# =====  
  
if __name__ == "__main__":  
    # Setup Argument Parser  
    parser = argparse.ArgumentParser(description="ETS2 Telemetry to IPC Bridge")  
    parser.add_argument("--ip", default="10.10.129.118", help="ETS2 Server IP")  
    args = parser.parse_args()  
  
    # Create Config  
    config = Config()  
    config.TELEMETRY_URL = "http://10.10.129.118:25555/api/ets2/telemetry"  
    # Run Application  
    with ETSTelemetryMonitor(config) as app:  
        app.start()
```

Control App 실행

- Ctrl + s 로 저장
- \$ cd Fabless_Education_blink/Step1
- \$ sudo python3 Data_parsing.py

```
topst@TOPST:~$ python3 Data_parsing.py
[INFO] Start polling: http://10.10.132.1:25555/api/ets2/telemetry
[INFO] Press Ctrl+C to stop.
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
rpm= 549 | speed_kmh= 0 | fuel_l= 791.03 | steering=-0.000
```

- 30fps로 값을 받아 출력
- RPM, 속도, 연료량, 휠 상태 외에도 여러 값을 수신해 활용이 가능

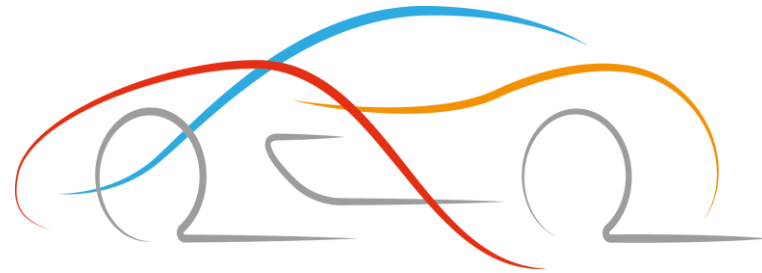
Control App 추가 기능 구현

- 기존에 작성했던 코드를 참고해 RPM, 속도, 연료량, 휠 외에 추가 적인 기능 구현
- EX) 브레이크 등, 비상등, 와이퍼 등
- 원하는 기능 추가해보기

- **Telemetry Information through Rest API**

- 웹 주소 창에 'http://\${pc-ip4-address}:25555/api/ets2/telemetry' 입력
- 그림과 같이 웹페이지에 차량 상태 정보 출력
- 해당 내용을 확인하여 출력하고자 하는 기능 추가

```
{"game":{"connected":true,"gameName":"ETS2","paused":false,"time":"0001-01-04T02:17:00Z"},  
["id":"daf","make":"DAF","model":"XF","speed":0.000665952743,"cruiseControlSpeed":0.0,"  
engineRpmMax":2200.0,"fuel":1248.27466,"fuelCapacity":1400.0,"fuelAverageConsumption":0.1  
186776921,"wearWheels":0.029407274,"userSteer":0.0,"userThrottle":0.0,"userBrake":0.0,  
,"retarderBrake":0,"retarderStepCount":3,"parkBrakeOn":true,"motorBrakeOn":false,"brake  
PressureWarningOn":false,"airPressureWarningValue":69.6,"airPressureEmergencyOn":false,"  
erTemperature":16.2492638,"waterTemperatureWarningOn":false,"waterTemperatureWarningVali  
dOn":true,"blinkerLeftActive":false,"blinkerRightActive":false,"blinkerLeftOn":false,"l  
BeaconOn":false,"lightsBrakeOn":false,"lightsReverseOn":false,"differentialRatio":0.0,"}
```



Thank You